NATIONAL ECONOMICS UNIVERSITY, HANOI
COLLEGE OF TECHNOLOGY
**FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE**


## ADVANCED DATABASE

**REPORT**

# High-Performance Crypto Tracker & AI Assistant:

## A Microservices Architecture Approach

**Instructor:** M.Sc. LE DUY KHANH

| **Students:** | NGUYEN TRONG DAI | 11247268 |
|---|---|---|
| | NGUYEN NGAN AN | 11247253 |
| | MAI HUY DANG | 11247269 |
| | MAI TUAN MANH | 11247318 |

Source Code: https://github.com/Dai-Nguyen1506/stock-tracker

Hanoi, May 2026

# Contents

# List of Figures

# List of Tables

# 1 Project Overview

## 1.1 Background and Motivation

The advent of digital assets and decentralized finance has catalyzed a paradigm shift in global financial markets. Unlike traditional stock exchanges that operate within strict business hours, the cryptocurrency market operates continuously, twenty-four hours a day, seven days a week. This continuous operation, coupled with the high volatility inherent to digital assets, generates an unprecedented volume of high-velocity time-series data. Every second, thousands of trades, order book updates, and market fluctuations are broadcasted across various cryptocurrency exchanges.

For modern financial applications, the ability to capture, process, and analyze this data in real-time is not merely a competitive advantage but a fundamental requirement. However, this poses significant challenges from a database management perspective. Traditional Relational Database Management Systems (RDBMS) are typically optimized for transactional integrity (ACID properties) and structured relational queries, rather than the relentless, high-throughput ingestion of append-only time-series data. When subjected to the load of tens of thousands of inserts per second, conventional databases often experience severe bottlenecks, resulting in unacceptable latency that renders real-time tracking ineffective.

Therefore, the primary objective of this project is to construct a system capable of handling large-scale data with ultra-low latency. This necessitates a departure from monolithic database architectures towards distributed, specialized database ecosystems. By exploring the fundamental differences in read/write mechanisms, indexing strategies, and clustering methodologies between NoSQL and SQL databases, this project seeks to provide a practical, high-performance solution to the data ingestion bottleneck prevalent in modern algorithmic trading and market tracking platforms.

## 1.2 Project Introduction

Addressing the challenges outlined above, this project introduces the "High-Performance Crypto Tracker & AI Assistant". This system is designed as a comprehensive financial market tracking platform that processes tens of thousands of data records per second while maintaining ultra-low latency.

At its core, the project is an applied exploration of database optimization, distributed systems, and modern software architecture. It moves beyond theoretical concepts by implementing a robust data pipeline that ingests live data streams from major

cryptocurrency exchanges and news providers. Furthermore, the system elevates the user experience by integrating an Artificial Intelligence (AI) Assistant utilizing Retrieval-Augmented Generation (RAG) technology. This AI component bridges the gap between raw quantitative data and qualitative market sentiment, providing users with context-aware insights.

The project is built upon a microservices architecture and is fully containerized using Docker, ensuring that the complex interactions between various database layers and application services are isolated, scalable, and easily deployable across different environments [4].

## 1.3 Key Features and Capabilities

The system is distinguished by several core functionalities, each demonstrating advanced concepts in data handling and application design:

- **Real-time Data Ingestion:** The system actively tracks and ingests data for over 400 cryptocurrency pairs continuously via WebSockets from Binance [2], alongside real-time financial news streams from Alpaca [1]. This feature demonstrates the system's capacity to handle massive concurrent data streams without data loss.
- **Ultra-fast Processing with Apache Cassandra:** To conquer the data ingestion bottleneck, the system utilizes Apache Cassandra [10] as its primary time-series database. By leveraging the Acsylla library [5], a C++ driver wrapper, the system achieves the capability to write tens of thousands of records in mere milliseconds. This highlights the superiority of distributed NoSQL databases in write-heavy workloads.
- **AI Financial Assistant (RAG Integration):** The project features an intelligent chatbot powered by Retrieval-Augmented Generation (RAG). When queried, the AI retrieves vectorized news data from ChromaDB [3] and real-time pricing data from Cassandra [10] to formulate accurate, context-driven investment advice and market summaries.
- **Interactive User Dashboard:** The frontend provides a professional-grade interface featuring Candlestick charts (using Lightweight Charts) and a live Orderbook that updates at intervals as frequent as 100 milliseconds. It also supports infinite scrolling, smoothly fetching historical data from the databases as the user navigates past timeframes.

- **Database Performance Benchmarking:** A unique feature of this project is its built-in tool for direct performance measurement and benchmarking. The system allows users to execute real-time comparative tests between Apache Cassandra and PostgreSQL, providing empirical evidence of their respective read and write speeds under heavy loads.

## 1.4 Architectural Overview and Database Ecosystem

To achieve the aforementioned capabilities, the system abandons the single-database approach in favor of a Multi-tier Database Layer, where different database technologies are utilized based on their specific architectural strengths.

The system architecture is divided into three primary tiers: the Frontend UI, the Ingestion Workers (running background data collection processes), and the Backend API. Connecting these application tiers is a sophisticated database ecosystem comprising four distinct technologies:

1. **Apache Cassandra:** Serving as the primary time-series database. It is mathematically modeled using Date Bucketing and Clustering Keys to store vast amounts of Klines (candlesticks), Orderbook depth, and news data, optimized for sequential disk writing and rapid retrieval [10].
2. **PostgreSQL:** Functioning as the benchmarking and control database. It is utilized to store parallel datasets, allowing the system to empirically compare the performance of a traditional relational architecture against the distributed NoSQL architecture of Cassandra [11].
3. **Redis:** Acting as an in-memory cache and message broker. It is responsible for sharing high-speed global metrics, such as real-time ingestion latency and transaction speeds, between the background workers and the API [9].
4. **ChromaDB:** Operating as a specialized Vector Database. It stores the semantic embeddings of incoming financial news, enabling the AI Assistant to perform high-speed similarity searches for the RAG pipeline [3].

By orchestrating these diverse database technologies within a cohesive Dockerized environment, the project serves as a comprehensive case study in modern data engineering, demonstrating how to architect systems capable of thriving in the demanding landscape of the cryptocurrency market.

# 2 Technology Stack and System Architecture

To ensure systemic robustness and verify that all performance requirements are met, the data pipeline was constructed and thoroughly validated prior to UI implementation. The overall system architecture is strictly decoupled into four primary components: Data Sources, Database Ecosystem, Backend Services, and Frontend.

## 2.1 Data Sources and Discovery Mechanism

The data ingestion layer relies on two primary modalities: WebSockets for real-time streaming and REST APIs for historical data retrieval. A critical component of this layer is the automated symbol discovery mechanism (implemented via `discovery.py`).

Upon initialization, the system fetches all supported cryptocurrency symbols from the Binance API [2] and cross-references them with symbols supported by the Alpaca API [1]. This creates two distinct tracking groups:

- **Intersected Group:** Symbols supported by both platforms. For these, the system subscribes to real-time Trades, Orderbook Depth, and Financial News streams.
- **Binance-Exclusive Group:** The remaining symbols. The system subscribes only to Trades and Orderbook Depth. This massive influx of concurrent data is intentionally utilized to stress-test and benchmark the write performance of Apache Cassandra [10].

All WebSocket subscriptions and initial payload parsings operate autonomously as background worker processes within the backend layer.

## 2.2 Multi-Tier Database Ecosystem

Once ingested, the high-velocity data is processed concurrently and routed to specialized databases tailored for specific data structures:

- **Redis (In-memory Buffer):** Acts as a high-speed message broker [9]. It absorbs the immediate influx of live WebSocket payloads and queues them for distribution to the Frontend, preventing data loss during traffic spikes.
- **Apache Cassandra (Time-Series Storage):** Serves as the heavy-duty storage engine [10]. It sequentially records calculated Candlesticks (Klines) derived

from live trades for historical querying, alongside raw Orderbook data utilized for continuous write-performance benchmarking.

- **ChromaDB (Vector Storage):** Dedicated to the AI Assistant pipeline [3]. It ingests the financial news from Alpaca, vectorizes the text along with its corresponding crypto symbol, and stores the embeddings for rapid semantic retrieval (RAG).

## 2.3 Backend Architecture and Auto-Recovery

The Backend, developed with FastAPI [8], orchestrates the core business logic and is functionally partitioned into three main pillars:

1. **Real-time Ingestion & Distribution:** Instantly processing incoming WebSocket payloads from external providers and securely writing them into the respective databases (Cassandra, Redis, ChromaDB).
2. **Data Retrieval & Benchmarking:** Providing RESTful endpoints primarily designed to serve the UI dashboard. Crucially, this includes the benchmarking routes that execute direct read/write comparisons between Cassandra and PostgreSQL [11].
3. **Data Recovery (Auto-Backfill):** A resilient synchronization mechanism. Upon system startup or during a timeframe navigation on the UI, this module detects temporal gaps in Cassandra's records (due to system downtime). It automatically triggers historical API calls to Binance to seamlessly backfill the missing data before rendering it to the user.

## 2.4 Frontend Decoupling via Internal WebSockets

A major architectural challenge in tracking over 400 concurrent cryptocurrency pairs is preventing the Frontend from crashing under the weight of thousands of direct external WebSocket connections.

To resolve this, the architecture implements a decoupled distribution model. External data is first pushed into Redis. The Backend then opens a unified, internal WebSocket channel directly to the Frontend React application [7]. By consolidating the data stream and utilizing Redis as a distribution buffer, the system significantly reduces the computational burden on the client's browser. This architectural choice guarantees that the UI can smoothly and continuously render highly volatile

candlestick charts and orderbooks across hundreds of symbols without experiencing rendering lag or memory overload.

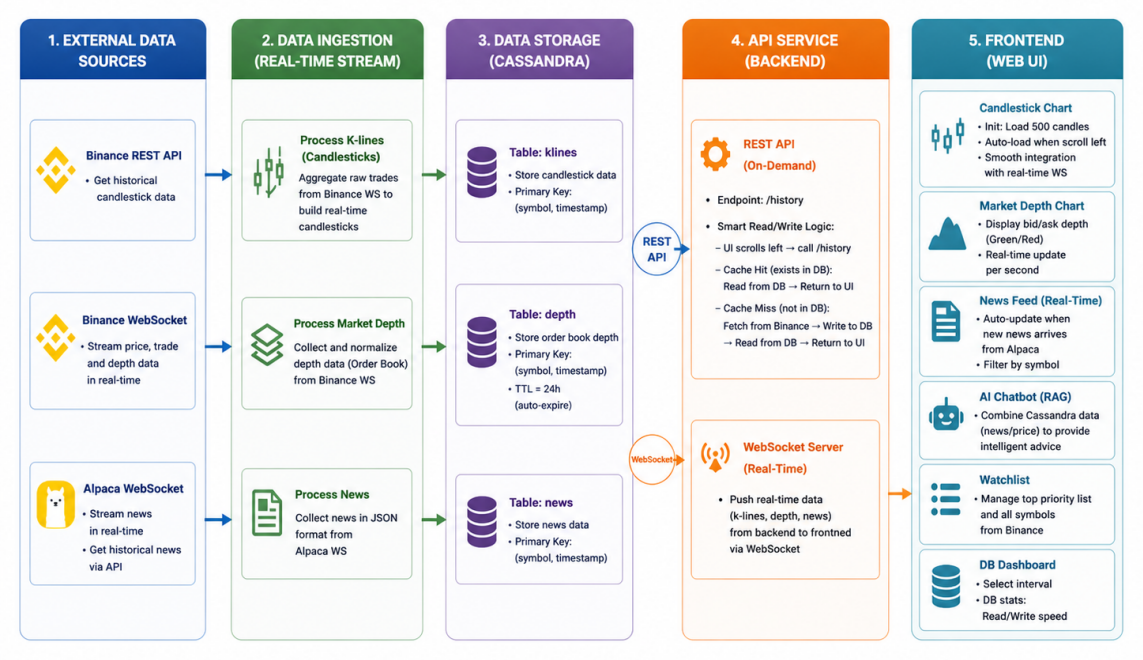


Figure 1: System Architecture and Data Flow Diagram

# 3 Database Design

## 3.1 Apache Cassandra

Apache Cassandra is an open-source NoSQL distributed database trusted by thousands of companies for its scalability and high availability without compromising performance. Its linear scalability and proven fault-tolerance on commodity hardware or cloud infrastructure make it the perfect platform for mission-critical data [10].

Because the primary data for this system consists of candlestick (Kline) data, the database schema is optimized specifically for time-series queries. The table utilizes a composite primary key structure. The Partition Key comprises the `symbol` (cryptocurrency ticker), `interval` (timeframe), and `date_bucket`. The inclusion of `date_bucket` is a crucial optimization technique designed to distribute data evenly across nodes and prevent the "wide partition" problem. Additionally, `timestamp` is utilized as the Clustering Key to sort records chronologically on the physical disk, which significantly accelerates the retrieval of historical data.

Furthermore, an auxiliary `orderbook` table was constructed to rigorously benchmark Cassandra's write performance. Similar to the Klines table, its Partition Key consists of `symbol` and `date_bucket`, with `timestamp` acting as the Clustering Key to maintain sequential ordering. Because this table is designed to absorb a massive, continuous influx of real-time WebSocket payloads, a Time-To-Live (TTL) mechanism was implemented. The system is configured to automatically expire and delete orderbook records after one hour, effectively preventing storage bloat and freeing up resources.

## 3.2 Redis

Redis is an open-source, in-memory data structure store that enables data retrieval with ultra-low latency, often operating in the microsecond range [9]. Within this system architecture, Redis serves as a critical high-speed buffer. It is deployed specifically to absorb the heavy, continuous influx of live data streaming from external WebSockets. By capturing this data in memory, Redis facilitates the rapid and seamless distribution of market updates directly to the frontend interface, preventing bottlenecks and reducing the load on the primary storage databases.

## 3.3 ChromaDB

ChromaDB is an open-source vector database specifically engineered to store and query embeddings for artificial intelligence applications [3]. Rather than storing data in traditional tabular or raw text formats, ChromaDB converts information into multidimensional numerical vectors. This architecture enables Semantic Search—the ability to locate information based on contextual and underlying meaning rather than relying strictly on exact keyword matches. This capability is highly beneficial for Retrieval-Augmented Generation (RAG) systems, allowing them to retrieve broader, more comprehensive, and contextually accurate data compared to standard databases. In this project, ChromaDB acts as the central repository for vectorizing and storing all market news, empowering the AI Assistant to perform advanced market analysis.

## 3.4 PostgreSQL

PostgreSQL is a highly robust, open-source relational database management system renowned for its durability, data integrity, and strict adherence to ACID properties [11]. While relational databases are the industry standard for transactional data, they can face limitations when ingesting massive volumes of time-series data. To empirically demonstrate the necessity of Cassandra in this high-frequency tracking system, PostgreSQL was integrated to serve as a comparative benchmark baseline. To ensure a fair and accurate performance comparison, the PostgreSQL tables were structurally architected to closely mirror the schemas utilized in Cassandra.

## 3.5 Performance Optimization Techniques

To meet the stringent requirements of high-frequency financial data processing and ultra-low latency, the system implements a comprehensive suite of optimization strategies. These enhancements span from the application layer down to physical storage, strictly adhering to Apache Cassandra production standards:

- **Communication Layer Optimization (High-Performance Driver):** Instead of relying on a native, pure-Python database driver, the project integrates the `acsylla` library [5]—an asynchronous wrapper built upon the robust DataStax C++ Driver. By executing I/O operations closer to the operating system level, this approach significantly mitigates Python's inherent execution overhead, accelerating processing speeds by an order of magnitude and highly optimizing system resources.

- **Bulk Data Ingestion Strategy (Memory Buffer & Batching):** The system employs an in-memory buffering mechanism, accumulating incoming data in RAM and executing a `flush()` operation periodically (every 60 seconds). This strategy is coupled with the `UNLOGGED BATCH` technique, strictly applied per Partition Key. This method aggregates hundreds of records into a single execution unit, eliminating the overhead of redundant transaction logs and drastically optimizing disk write bandwidth.
- **Parallel Concurrency Handling:** Leveraging the power of asynchronous programming via Python's `asyncio` coupled with a Semaphore control (limit set to 500), the system safely sustains hundreds of concurrent write requests. This mechanism fully exploits the multi-core processing capabilities of the Cassandra cluster and maximizes network throughput, effectively preventing any data bottlenecks at the ingestion worker level.
- **Query Optimization (Prepared Statements):** All data insertion operations strictly utilize Prepared Statements. This architectural choice ensures that the query structure is compiled and cached on the server side prior to execution, thereby conserving CPU resources and shortening the response time for subsequent, repetitive statement executions.
- **Storage Management and Data Compression (Storage Efficiency):** To handle the massive scale of historical data, the physical storage layer is optimized through two primary methods:
  - *TimeWindowCompactionStrategy (TWCS):* The implementation of this specialized compaction algorithm is tailored for time-series workloads. It efficiently manages SSTable files within defined time windows and seamlessly automates the purging of expired data governed by Time-To-Live (TTL) parameters.
  - *LZ4 Compression:* The activation of the LZ4 compression standard at the table level minimizes the physical storage footprint. It optimizes Disk I/O performance by reducing the amount of bytes written to the disk without imposing a heavy processing burden on the CPU.

# 4 Performance Benchmarking: Cassandra vs. PostgreSQL

## 4.1 Benchmarking Methodology

A distinguishing feature of this system is its built-in evaluation framework, allowing for real-time comparative analysis between a distributed NoSQL database (Apache Cassandra) and a traditional Relational Database (PostgreSQL) [10, 11]. The objective of this benchmark is to empirically evaluate their performance under the extreme I/O loads typical of high-frequency cryptocurrency tracking.

The benchmarking suite executes two primary tests via the frontend dashboard:

- **Write Performance (Bulk Ingestion):** Measuring the time required to flush a batch of approximately 40,000 concurrent records (comprising candlesticks and orderbook depth) from RAM to the physical disk.
- **Read Performance (Historical Query):** Measuring the latency of retrieving a large dataset of historical time-series data (approximately 90,000 candlestick rows) mimicking a user panning the UI chart deep into the past.

## 4.2 Write Performance Analysis (Ingestion)

The system employs advanced optimization techniques for both databases. PostgreSQL utilizes the `asyncpg` library with C-level binary COPY execution for bulk inserts, while Cassandra leverages the `acsylla` C++ driver to execute parallel, unlogged batches [5].

Table 1 illustrates the results over five consecutive test iterations.

**Analytical Conclusion:** Apache Cassandra demonstrates overwhelming superiority in write-heavy workloads. On average, Cassandra completed the ingestion nearly six times faster than PostgreSQL. This exponential performance gain is attributed to Cassandra's Log-Structured Merge-tree (LSM) architecture and the utilization of asynchronous parallel concurrency [10]. While PostgreSQL performs admirably given its relational constraints (averaging roughly 900ms), it is fundamentally bottlenecked by its sequential write architecture and the overhead of maintaining ACID transactional logs [11].

| Test Iteration | Cassandra (Parallel Batch) | PostgreSQL (Bulk Insert) |
|---|---|---|
| 1 | 116.50 ms | 1064.03 ms |
| 2 | 381.37 ms | 1081.91 ms |
| 3 | 105.53 ms | 874.58 ms |
| 4 | 93.02 ms | 776.52 ms |
| 5 | 104.06 ms | 792.00 ms |
| **Average** | **$\sim$ 160.10 ms** | **$\sim$ 917.80 ms** |

Table 1: Write Performance Comparison for $\sim$ 40,000 records.

## 4.3 Read Performance Analysis (Retrieval)

High read throughput is equally critical to ensure the frontend dashboard renders smoothly without latency spikes. Table 2 summarizes the retrieval times for a batch query of roughly 90,000 rows.

| Test Iteration | Cassandra | PostgreSQL |
|---|---|---|
| 1 | 102 ms | 185 ms |
| 2 | 93 ms | 158 ms |
| 3 | 118 ms | 141 ms |
| 4 | 111 ms | 119 ms |
| 5 | 89 ms | 126 ms |
| **Average** | **$\sim$ 102.6 ms** | **$\sim$ 145.8 ms** |

Table 2: Read Performance Comparison for querying $\sim$ 90,000 historical records.

**Analytical Conclusion:** Cassandra maintains a highly stable and superior read latency, averaging approximately 102.6 ms compared to PostgreSQL's 145.8 ms. The architectural justification for this lies in Cassandra's schema design. By defining a `Clustering Order` mathematically descending by timestamp, the most recent data is sorted sequentially at the physical disk level [10]. Consequently, querying the latest 90,000 records merely requires a continuous disk scan. Conversely, PostgreSQL must often load data into RAM and execute sorting operations computationally, incurring higher latency. While PostgreSQL's read speed slightly improves in later iterations due to its Shared Buffers caching mechanism [11], it remains consistently slower than

Cassandra.

## 4.4 Final Verdict

The empirical benchmarking conclusively validates the multi-tier database architectural choice. Apache Cassandra is unequivocally the optimal solution for managing the real-time ingestion firehose and serving rapid historical data for the frontend. PostgreSQL, while highly reliable, serves effectively as a secondary or benchmarking anchor but lacks the raw I/O velocity required for a high-frequency financial tracking core.

# 5 Frontend Architecture and Implementation

## 5.1 Technology Stack and Codebase Structure

The user interface is engineered as a highly responsive Single Page Application (SPA). To ensure rapid development and optimized build processes, the frontend is bundled using Vite. Crucially, the entire frontend codebase is implemented in **TypeScript** (evidenced by the extensive use of `.tsx` and `.ts` files), which enforces strict type safety, reduces runtime errors, and enhances the maintainability of complex financial data structures.

The project adopts a modular, component-based architecture. Global styles and modular CSS are managed through files like `index.css` and `App.module.css`, ensuring a scalable and collision-free styling approach across the dashboard.

## 5.2 Real-time Data Streaming via Custom Hooks

A significant architectural achievement in the frontend implementation is the decoupling of WebSocket communication logic from the rendering components. Instead of cluttering UI components with connection states, the system utilizes custom React hooks encapsulated within the `src/hooks/` directory.

Specifically, the system implements:

- `useBinanceKlineStream.ts`: A dedicated hook engineered to manage the continuous stream of Candlestick (Kline) data directly from Binance [2]. It handles the ingestion, state updates, and potential reconnection logic for the time-series price data.
- `useStockWebSocket.ts`: A secondary custom hook designed to manage broader market data streams, ensuring that multiple WebSocket connections can operate concurrently without blocking the main browser thread.

## 5.3 Core Visualization Components

The visual representation of the high-velocity data is handled by specialized components within the `src/components/` directory, each mapped to a specific market feature:

- `MainChart.tsx`: This component acts as the primary viewport for historical and real-time price action. It consumes the data provided by the Kline stream

hooks and renders the interactive candlestick chart, handling user inputs such as panning and timeframe selection.

- `DepthChart.tsx`: Rendering the market orderbook requires handling rapid, sub-second state changes. This dedicated component visualizes the bid and ask liquidity (market depth), translating complex JSON payloads into an intuitive, dual-sided graphical format.

## 5.4 Dashboard Layout and Modularity

To manage the dense concentration of information inherent to trading platforms, the application's layout is strategically divided. The main structural components, alongside the central charts, include `SidebarLeft.tsx` and `SidebarRight.tsx`.

These sidebar components encapsulate peripheral but essential functionalities, such as the AI Assistant chat interface, real-time news feeds, and the Database Benchmarking control panel. By isolating these features into distinct layout modules, the React [7] application prevents unnecessary re-renders of the heavy charting components when user interactions occur in the sidebars, thereby maintaining a smooth 60 Frames Per Second (FPS) rendering performance.

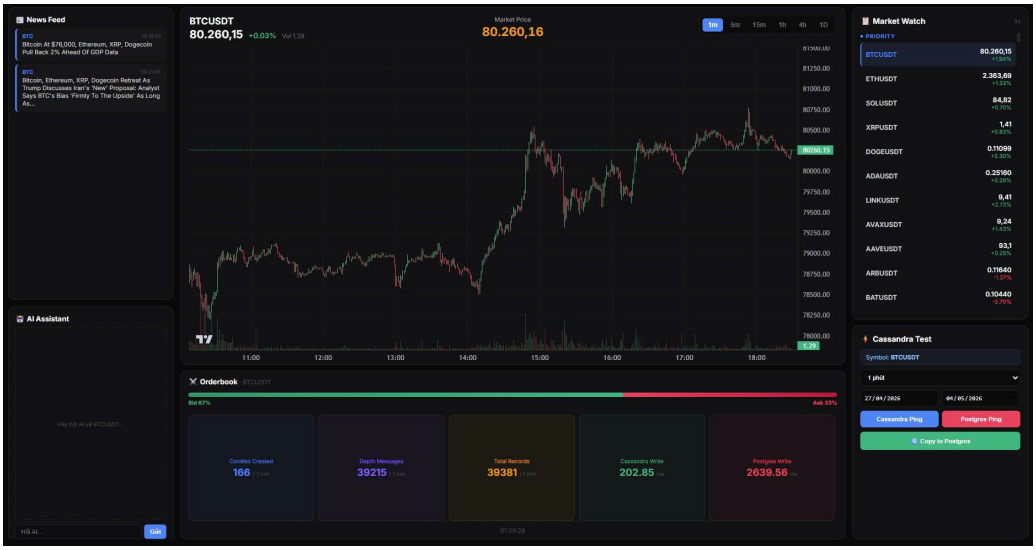


Figure 2: The High-Performance Trading Dashboard featuring real-time candlestick charts, orderbook depth, and AI Assistant.

# 6 Integration of the Financial AI Assistant

## 6.1 Retrieval-Augmented Generation (RAG) Architecture

A defining feature of this project is the integration of an intelligent Financial AI Assistant, elevating the platform from a mere data tracker to an analytical tool. To ensure the AI provides accurate, up-to-date, and contextually relevant financial insights, the system implements a Retrieval-Augmented Generation (RAG) architecture. Rather than relying solely on the pre-trained knowledge of a Large Language Model (LLM), the RAG pipeline dynamically fetches real-time market data and news before generating a response.

## 6.2 Vectorization and Context Retrieval

When a user submits a query (e.g., "What is the current situation of BTC?"), the backend executes a multi-step retrieval process:

- **Entity Extraction:** The system first parses the user's natural language input to identify specific cryptocurrency symbols.
- **Semantic Search via ChromaDB:** The system queries ChromaDB [3], the dedicated vector database, to retrieve the most contextually relevant and recent financial news embedded from the Alpaca WebSocket stream [1].
- **Quantitative Data Retrieval:** Simultaneously, the system queries Apache Cassandra [10] to fetch the latest price action and historical market data for the identified symbol.

This combination of qualitative news (from ChromaDB) and quantitative metrics (from Cassandra) is synthesized into a comprehensive prompt context.

## 6.3 LLM Integration and Response Generation

To process the augmented prompt and generate the final analysis, the system relies on the **Google Gemini API** [6] as its primary Large Language Model (LLM) engine. In financial applications, generating accurate and context-aware insights from a dense volume of quantitative and qualitative data is a critical requirement.

By utilizing Gemini, the RAG pipeline benefits from advanced reasoning capabilities and a highly stable, production-ready infrastructure. The system seamlessly routes the synthesized context—comprising vectorized news from ChromaDB and historical

price metrics from Cassandra—directly to the Gemini model. This streamlined orchestration ensures high reliability and uninterrupted conversational service for the user, delivering precise market analyses efficiently without the architectural overhead of maintaining multiple fallback layers.

# 7 System Deployment and Containerization

## 7.1 Deployment Strategy

Deploying a complex microservices architecture—especially one involving a multi-tier database ecosystem and real-time ingestion workers—requires a rigorous, reproducible, and isolated deployment strategy. To address this, the entire "High-Performance Crypto Tracker & AI Assistant" system is fully containerized using Docker [4].

This containerization strategy guarantees that the system behaves consistently across different development, testing, and production environments, effectively eliminating the "it works on my machine" dilemma. It encapsulates the intricate dependencies of Python (for the backend), React (for the frontend), and the various database engines into standardized, isolated units.

## 7.2 Container Orchestration with Docker Compose

The orchestration of these isolated containers is managed via a centralized `docker-compose.yml` file [4]. This declarative configuration file defines the network topologies, volume mounts, and startup sequences for all services within the project.

The deployment ecosystem is structured into several interconnected services:

- **Frontend Service:** The ReactJS application is built and served within its own container, utilizing the `frontend/Dockerfile`. It is exposed on port `5173` to provide the user interface.
- **Backend API Service:** The FastAPI application is containerized using the `backend/Dockerfile`. It handles incoming HTTP requests and internal WebSockets, connecting the frontend to the underlying databases.
- **Ingestion Workers:** Separate container instances are spawned for the Binance and Alpaca WebSocket listeners, ensuring that the heavy I/O operations of data collection do not block the main API threads.
- **Database Infrastructure:** Official Docker images are utilized for Apache Cassandra, PostgreSQL, Redis, and ChromaDB [10, 11, 9, 3].

## 7.3 Persistent Storage and Initialization

A critical aspect of database deployment is ensuring data persistence across container restarts. The `docker-compose.yml` configuration maps internal container directories to physical Host volumes. For example, the ChromaDB vector embeddings and AI model caches are mounted to a local `./.cache` directory, preventing the need to re-download massive models upon every startup.

Furthermore, the deployment pipeline incorporates an automated initialization process. A dedicated `cassandra-init` container is triggered during the first launch. This transient container executes the `cassandra/init.cql` script to automatically create the necessary keyspaces, tables, and clustering structures (such as `klines`, `news`, and `orderbook`) before the backend services are allowed to connect.

## 7.4 Execution Runbook

The deployment process has been streamlined to require minimal manual intervention. To deploy the entire ecosystem, the operator simply executes the following command from the project root:

```
docker compose up -d --build
```

This single command instructs Docker [4] to build the custom images for the frontend and backend, pull the required database images, establish the internal bridge network, and launch the services in detached mode (`-d`).

Once the deployment sequence is complete, operators can monitor the real-time data ingestion streams by inspecting the container logs (e.g., `docker compose logs ingestion-binance -f`).

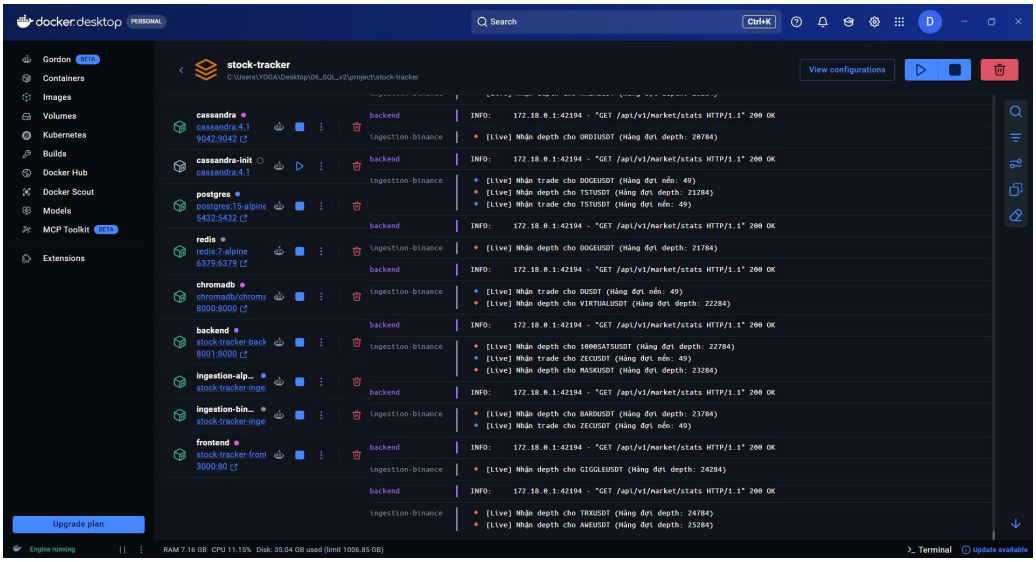


Figure 3: Containerized microservices running successfully within the Docker environment.

# 8 Conclusion and Future Work

## 8.1 Conclusion

This report has detailed the design and implementation process of a high-performance cryptocurrency market monitoring system integrated with a financial AI assistant. The system was engineered to solve the critical challenge of real-time data processing with high throughput and ultra-low latency—a fundamental requirement in the continuously operating and highly volatile cryptocurrency market.

Throughout its development and deployment, the project successfully met its core objectives, demonstrated by the following key achievements:

- **High-Performance Time-Series Storage:** Apache Cassandra [10] proved to be a highly suitable choice as the primary time-series database. Its write-optimized architecture and horizontal scalability effectively handled the massive, continuous influx of real-time data streams. Structuring data utilizing Partition Keys and Clustering Keys facilitated highly efficient time-based queries, allowing the system to maintain stable write speeds and low latency even as data volumes rapidly scaled.
- **Specialized Multi-Tier Database Ecosystem:** Alongside Cassandra, the project leveraged a diverse database ecosystem to support specific architectural needs. Redis [9] acted as a critical high-speed buffer and message broker for live data streams, while ChromaDB [3] managed vector embeddings to enable semantic querying within the AI pipeline. Concurrently, PostgreSQL [11] served as a reliable benchmarking baseline, empirically highlighting Cassandra's superiority in handling large-scale write operations.
- **Seamless Real-Time Data Integration:** The system effectively integrated live data feeds from Binance [2] and Alpaca [1], enabling the simultaneous ingestion of trade data, orderbook depth, and financial news. This consolidated stream not only powered the real-time UI but also established a robust foundational dataset for the AI analytics engine.
- **Optimized Application Layers:** At the backend, the asynchronous model built with FastAPI [8] demonstrated exceptional capability in processing continuous data streams with minimal latency. On the client side, the frontend developed with React [7], utilizing WebSocket connections and custom hooks, ensured the smooth and responsive rendering of real-time data, successfully handling hundreds of concurrent trading pairs without performance degrada-

tion.

Ultimately, the project delivered a robust foundational architecture capable of comprehensive performance benchmarking. This solid baseline paves the way for integrating more complex features, such as advanced AI assistants, deep financial analytics, and automated algorithmic trading systems.

## 8.2 Future Work

Despite achieving highly positive results and establishing a stable core architecture, there are several promising avenues for future development and system enhancement:

- **Enhancing the AI Engine:** While the current system utilizes a Retrieval-Augmented Generation (RAG) pipeline with a generalized Large Language Model (LLM), future iterations could significantly improve analytical accuracy by employing specialized, fine-tuned LLMs trained explicitly on financial corpora. Additionally, integrating diverse data pipelines—such as social media sentiment, on-chain metrics, and macroeconomic indicators (e.g., interest rates, CPI)—would empower the AI Assistant to generate deeper and more holistic market insights.
- **Advanced Predictive Analytics:** Currently, the platform focuses primarily on real-time tracking and visualization. Future work involves developing and deploying Machine Learning and Deep Learning models (such as LSTMs or Transformer-based architectures for time-series forecasting) to predict short-term price trends, detect market anomalies, and perform automated risk assessments.
- **Database Infrastructure Optimization:** Managing a multi-tier database ecosystem introduces querying complexity. Researching and implementing a unified query engine or establishing a centralized Data Lake could streamline operations across different database technologies. Furthermore, continuous refinement of schemas, indexing mechanisms, and caching strategies will be vital to incrementally boost overall system throughput and query efficiency.
- **System Security and Compliance:** To transition the platform towards a production-ready financial application, robust security measures must be implemented. This includes integrating comprehensive user authentication protocols, strict Role-Based Access Control (RBAC), and end-to-end data encryption to ensure the safety and integrity of user information and system operations.

- **User Experience Customization:** Developing personalized features, such as automated portfolio recommendations and dynamically customizable dashboards tailored to individual user behaviors and trading preferences. Implementing recommendation algorithms will provide users with highly relevant information, significantly elevating the platform's utility and user engagement.

# References

[1] Alpaca Securities LLC. *Alpaca Trading API Documentation*. https://alpaca.markets/docs/. Accessed: 2026-05-05. 2026.

[2] Binance. *Binance Spot API Documentation*. https://binance-docs.github.io/apidocs/spot/en/. Accessed: 2026-05-05. 2026.

[3] Chroma Inc. *Chroma: The AI-native Open-Source Embedding Database*. https://www.trychroma.com/. Accessed: 2026-05-05. 2026.

[4] Docker Inc. *Docker: Accelerated Container Application Development*. https://www.docker.com/. Accessed: 2026-05-05. 2026.

[5] Pau Freixes. *Acsylla: A High Performance Asynchronous Cassandra Python Driver*. https://github.com/acsylla/acsylla. Accessed: 2026-05-05. 2026.

[6] Google DeepMind. *Google Gemini API*. https://ai.google.dev/. Accessed: 2026-05-05. 2026.

[7] Meta Platforms, Inc. *React: The Library for Web and Native User Interfaces*. https://react.dev/. Accessed: 2026-05-05. 2026.

[8] Sebastián Ramírez. *FastAPI Framework, High Performance, Easy to Learn, Fast to Code, Ready for Production*. https://fastapi.tiangolo.com/. Accessed: 2026-05-05. 2026.

[9] Redis Ltd. *Redis: The Open Source, In-Memory Data Store*. https://redis.io/. Accessed: 2026-05-05. 2026.

[10] The Apache Software Foundation. *Apache Cassandra: The Open Source NoSQL Database*. https://cassandra.apache.org/. Accessed: 2026-05-05. 2026.

[11] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. https://www.postgresql.org/. Accessed: 2026-05-05. 2026.